Euroleague Player Statistics Collection Guide

Replicating NBA Lineup Analysis for Euroleague (2010-2025)

Overview

This guide provides a complete pipeline for collecting Euroleague player statistics similar to the NBA Lineup Analysis paper by Kalman & Bosch. The data covers seasons 2010/2011 through 2024/2025.

Key Design Decision: European Metrics

- All measurements use **native European units** (meters, centimeters)
- No conversion to imperial units needed
- More accurate for Euroleague's actual court dimensions
- Shot zones defined in meters: 0-0.91m, 0.91-3.05m, 3.05m-6.75m
- Easier integration with other European basketball data sources

---

1. Installation & Setup

Required Packages

```bash
# Install the Euroleague API package
pip install euroleague-api

# Install additional dependencies
pip install pandas numpy scipy scikit-learn matplotlib seaborn
```

API Structure

The Euroleague API provides several endpoints:

- **Player Stats**: Basic and advanced statistics
- **Team Stats**: Team-level aggregations
- **Shot Data**: Spatial shooting data
- **Game Data**: Box scores and play-by-play
- **Standings**: League tables

2. Data Collection Strategy

Statistics Mapping: NBA Paper → Euroleague (European Metrics)

NBA Stat (Table 1)	Euroleague Equivalent	Metric Used	API Endpoint	Notes
Height	Height	cm	PlayerStats	Keep in centimeters
Offensive Rebound Rate	OReb / Total_Reb	percentage	PlayerStats	Calculate from totals
Defensive Rebound Rate	DReb / Total_Reb	percentage	PlayerStats	Calculate from totals
Assist Rate	AST / FGM	rate	PlayerStats	Approximation
Steal Rate	STL per game	per game	PlayerStats	Direct mapping
Block Rate	BLK per game	per game	PlayerStats	Direct mapping
Turnover Rate	TO per 100 poss	per 100	Per100Possessions	Direct mapping
Points	PTS per 100 poss	per 100	Per100Possessions	Direct mapping
Usage Rate	Calculated	percentage	PlayerStats	(FGA + 0.44*FTA + TO) / Team_Poss
Player Efficiency Rating	PIR	rating	PlayerStats	Euroleague's efficiency metric
Free Throw Rate	FTM / FGA	rate	PlayerStats	Calculate from totals
Free Throw Percentage	FT%	percentage	PlayerStats	Direct mapping
Field Goals Attempted	FGA per 100	per 100	Per100Possessions	Direct mapping
2FG%	2P%	percentage	PlayerStats	Direct mapping
3FG%	3P%	percentage	PlayerStats	Direct mapping
2FG Assist Rate	Calculated	percentage	Shot data + PlayerStats	Requires shot tracking data
3FGA%	3PA / FGA * 100	percentage	PlayerStats	Calculate from totals

NBA Stat (Table 1)	Euroleague Equivalent	Metric Used	API Endpoint	Notes
Corner 3FGA%	From shot data	percentage	ShotData endpoint	Requires spatial analysis
3FG Assist Rate	Calculated	percentage	Shot data + PlayerStats	Requires shot tracking data
Dunk Attempt Rate	From shot data	percentage	ShotData endpoint	Limited availability
0-0.91m FGA%	From shot data	meters	ShotData endpoint	At rim zone (~0-3 ft)
0.91-3.05m FGA%	From shot data	meters	ShotData endpoint	Short midrange (~3-10 ft)
3.05m-3p FGA%	From shot data	meters	ShotData endpoint	Long midrange (~10ft-3p)

Shot Distance Zones (European Metrics):

- **0-0.91m**: At rim / Paint area
- **0.91-3.05m**: Short midrange
- **3.05m-6.75m**: Long midrange (to 3-point line)
- **6.75m**: Three-point line distance in Euroleague

---

3. Data Architecture

Recommended Directory Structure

```
euroleague_clustering_data/
│
├── raw/                        # Raw API responses
│   ├── season_2010_2011/
│   │   ├── basic_pergame.csv
│   │   ├── advanced_pergame.csv
│   │   ├── per100.csv
│   │   ├── accumulated.csv
│   │   └── game_metadata.csv
│   ├── season_2011_2012/
│   └── ...
│
├── processed/                  # Cleaned & standardized data
│   ├── euroleague_player_stats_2010_2024.csv
│   ├── euroleague_player_stats_with_shot_zones.csv
│   └── player_season_summary.csv
│
```

```
├── metadata/                    # Collection logs & documentation
│   ├── collection_log_20241229_120000.json
│   ├── variable_definitions.json
│   └── data_quality_report.txt
│
├── analysis/                    # Analysis outputs
│   ├── clustering/
│   │   ├── cluster_assignments.csv
│   │   ├── cluster_profiles.csv
│   │   └── cluster_visualization.png
│   ├── lineups/
│   │   ├── lineup_ratings.csv
│   │   └── lineup_predictions.csv
│   └── models/
│       ├── player_clustering_model.pkl
│       └── lineup_rating_model.pkl
│
└── notebooks/                   # Jupyter notebooks
    ├── 01_data_exploration.ipynb
    ├── 02_clustering_analysis.ipynb
    └── 03_lineup_modeling.ipynb
```

---

4. Data Collection Best Practices

Season Coverage

python

```python
# Adjust based on data availability
SEASONS = {
    'start': 2010,  # 2010-2011 season
    'end': 2024,    # 2024-2025 season
    'current_incomplete': True  # Adjust if mid-season
}

# Consider collecting by phase
PHASE_TYPES = {
    'RS': 'Regular Season',
    'PO': 'Playoffs',
    'FF': 'Final Four'
}
```

Rate Limiting & Error Handling

- Add 1-2 second delays between API calls
- Implement exponential backoff for failed requests
- Save progress incrementally (season by season)
- Log all errors with timestamps

Data Quality Filters

Following the NBA paper methodology:

```python
# Minimum games played threshold
MIN_GAMES = 15  # Euroleague season ~34 games (vs 82 in NBA)

# Minimum minutes per game
MIN_MINUTES = 5.0

# Remove duplicate entries
# Handle player trades (appears in multiple teams)
```

---

5. Handling Euroleague-Specific Considerations

Advantage: Native European Metrics

- **Height**: Centimeters (no conversion needed)
- **Court dimensions**: 28m × 15m
- **3-point line**: 6.75m (vs NBA's 7.24m)
- **Shot zones**: Natural metric distances (0-0.91m, 0.91-3.05m, 3.05m-6.75m)

Challenge 1: Shorter Seasons

- **NBA**: 82 games
- **Euroleague**: ~34 regular season games
- **Solution**: Adjust minimum games threshold proportionally (15 vs 30)

Challenge 2: Different Efficiency Metrics

- **NBA**: Player Efficiency Rating (PER)

- **Euroleague**: Performance Index Rating (PIR)
- **Solution**: Use PIR as direct equivalent (scale is different but concept is same)

Challenge 3: Limited Shot Tracking Data

- Shot location data is **less complete** than NBA SportVU
- Not all seasons have detailed spatial data
- **Solution**:
    - Use available shot data for recent seasons (2015+)
    - Approximate shot zones using 2P/3P splits for older seasons
    - Consider excluding zone variables if insufficient data

Challenge 4: Player Movement

- More international player movement between leagues
- Mid-season transfers
- **Solution**:
    - Track player-team-season combinations
    - Aggregate stats if player appears with multiple teams

Challenge 5: Court Dimensions Differences

- **Euroleague**: 28m × 15m, 3-point at 6.75m
- **NBA**: 28.65m × 15.24m, 3-point at 7.24m (corners: 6.71m)
- **Impact**: Slightly different shot distributions and spacing
- **Solution**: Use native Euroleague metrics without conversion

---

6. Data Storage & Format Recommendations

File Formats

For Raw Data:

- CSV files (human-readable, version control friendly)
- Separate files by data type and season

For Processed Data:

- CSV for tabular data (easy sharing)
- Parquet for large datasets (efficient storage/loading)
- PKL for Python-specific objects (models, complex structures)

For Metadata:

- JSON for logs and configurations
- YAML for pipeline parameters

Database Option (Optional)

For production pipelines, consider PostgreSQL/SQLite:

```sql
-- Schema example
CREATE TABLE player_stats (
    player_id VARCHAR(50),
    player_name VARCHAR(100),
    season VARCHAR(10),
    team VARCHAR(50),
    games_played INT,
    minutes_played FLOAT,
    points_per100 FLOAT,
    usage_rate FLOAT,
    -- ... all Table 1 variables
    PRIMARY KEY (player_id, season, team)
);

CREATE TABLE shot_locations (
    player_id VARCHAR(50),
    season VARCHAR(10),
    shot_zone VARCHAR(50),
    attempts INT,
    makes INT,
    PRIMARY KEY (player_id, season, shot_zone)
);
```

---

7. Usage Examples

Basic Collection

```python
```

```python
from euroleague_data_collector import EuroleagueDataCollector

# Initialize
collector = EuroleagueDataCollector(
    competition_code="E",
    output_dir="euroleague_data"
)

# Collect all seasons
df = collector.collect_all_seasons(
    start_season=2010,
    end_season=2024
)
```

Custom Analysis

python

```python
# Load processed data
import pandas as pd

df = pd.read_csv('euroleague_data/processed/euroleague_player_stats_2010_2024.csv')

# Filter for recent seasons
recent_df = df[df['Season'].isin(['2022-2023', '2023-2024', '2024-2025'])]

# Prepare for clustering (scale variables)
from sklearn.preprocessing import StandardScaler

# Features using European metrics
features = [
    'Height_cm',          # Height in centimeters
    'OReb_Rate', 'DReb_Rate', 'Assist_Rate',
    'Steal_Rate', 'Block_Rate', 'Turnover_Rate',
    'Points_Per100', 'Usage_Rate', '3FGA_Pct',
    'Player_Efficiency_Rating', 'FTr', 'FT_Pct',
    'FGA_Per100', '2FG_Pct', '3FG_Pct',
    '2FG_Assist_Rate', 'Corner_3FGA_Pct', '3FG_Assist_Rate',
    'Dunk_Attempt_Rate',
    '0-0.91m_FGA_Pct',    # At rim (metric)
    '0.91-3.05m_FGA_Pct',  # Short midrange (metric)
    '3.05m-3p_FGA_Pct'    # Long midrange (metric)
]

scaler = StandardScaler()
scaled_features = scaler.fit_transform(recent_df[features])
```

---

8. Next Steps: Clustering Analysis

After collecting data, replicate the NBA paper's methodology:

1. Model-Based Clustering

```python
```

```
from sklearn.mixture import GaussianMixture
from sklearn.model_selection import GridSearchCV

# Test different numbers of clusters
n_clusters_range = range(5, 12)
bic_scores = []

for n in n_clusters_range:
    gmm = GaussianMixture(n_components=n, covariance_type='full')
    gmm.fit(scaled_features)
    bic_scores.append(gmm.bic(scaled_features))

# Select optimal clusters using BIC
optimal_n = n_clusters_range[np.argmin(bic_scores)]
```

2. Cluster Interpretation

Analyze each cluster's characteristics:

- Average stats for each position
- Notable players in each cluster
- Evolution of player clusters over time

3. Lineup Analysis

Extend to lineup effectiveness prediction:

- Collect 5-man lineup data
- Build "soft lineups" using cluster probabilities
- Use Random Forest to predict lineup net rating

---

9. Data Validation Checklist

Before analysis, verify:

- [ ] All seasons collected successfully
- [ ] No duplicate player-season records
- [ ] Minimum games filter applied
- [ ] Missing data handled appropriately
- [ ] Variables scaled correctly
- [ ] Shot zone data availability documented

- [ ] Collection logs saved
- [ ] Data dictionary created

---

10. Limitations & Considerations

Known Limitations

1. **Shot tracking data**: Less comprehensive than NBA (especially pre-2015)
2. **Lineup data**: May be harder to obtain in same detail as NBA
3. **Roster changes**: More frequent mid-season in Euroleague
4. **Competition level**: Mix of skill levels across teams greater than NBA

Recommended Adjustments

- Focus on **2015-2025** for most complete data
- Use **PIR instead of PER** for efficiency
- Consider **team strength** as covariate
- **Cross-validate** with domain experts familiar with Euroleague

---

11. Resources & References

Documentation

- Euroleague API Documentation
- Euroleague Official Stats
- Original NBA Paper

Similar Work

- Analyzing European basketball with machine learning
- Shot chart analysis for Euroleague
- Tactical analysis of Euroleague teams

Contact & Contributions

For questions or improvements to this pipeline:

- Open issues on the GitHub repository
- Contribute additional endpoints or metrics
- Share your clustering results!

Last Updated: December 2024
Data Coverage: Seasons 2010/2011 - 2024/2025
API Version: euroleague-api 0.0.21